

Explain Kth Smallest Sums

Absolutely. This is the **Kth Smallest Element in a Sorted Matrix** problem, and the code you posted uses a **heap-based K-smallest-sums approach** rather than the more common **binary-search-on-answer** approach.

1. First understand the question

The problem is essentially:

You are given an $n \times n$ matrix where every row is sorted in ascending order. Find the **k-th smallest value** among all possible elements/values represented by the matrix.

For example:

```
mat =  
[  
  [1, 3, 11],  
  [2, 4, 6],  
  [5, 7, 8]  
]  
  
k = 5
```

If we simply collect all elements:

```
1, 3, 11, 2, 4, 6, 5, 7, 8
```

Sorted:

```
1, 2, 3, 4, 5, 6, 7, 8, 11
```

The 5th smallest is:

```
5
```

However, the code you posted is actually solving the **matrix k-th smallest sum** variant.

For example:

```
mat =  
[  
  [1, 10, 20],  
  [2, 3, 4],  
  [5, 6, 7]  
]  
  
k = 3
```

We choose **one element from each row**:

```
1 + 2 + 5 = 8  
1 + 2 + 6 = 9  
1 + 2 + 7 = 10  
1 + 3 + 5 = 9  
...
```

We need the **k-th smallest sum**.

Your code is designed for this problem.

2. The key intuition

Suppose:

```
mat =  
[  
  [1, 3, 5],  
  [2, 4, 6],  
  [7, 8, 9]  
]  
  
k = 3
```

Initially, we only have the first row:

```
sums = [1, 3, 5]
```

Think of these as:

```
"All possible sums after processing the first row."
```

Now process the second row.

We combine every existing sum with every element of the next row:

```
1 + 2 = 3
1 + 4 = 5
1 + 6 = 7
```

```
3 + 2 = 5
3 + 4 = 7
3 + 6 = 9
```

```
5 + 2 = 7
5 + 4 = 9
5 + 6 = 11
```

So:

```
all sums =
3, 5, 7, 5, 7, 9, 7, 9, 11
```

We only care about the **3 smallest**:

```
3, 5, 5
```

So instead of keeping all 9 values, we keep only:

```
sums = [3, 5, 5]
```

Then we process the third row.

Each current sum combines with:

```
7, 8, 9
```

Again we generate:

```
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12
```

```
5 + 7 = 12
5 + 8 = 13
5 + 9 = 14
```

```
5 + 7 = 12
5 + 8 = 13
5 + 9 = 14
```

Again, keep only the smallest $k = 3$:

```
10, 11, 12
```

Therefore the answer is:

```
12
```

3. Why do we keep only k sums?

This is the most important idea.

Suppose after processing some rows we have:

```
sums = [3, 5, 5, 10, 15, 20]
```

and:

```
k = 3
```

Do we need:

10, 15, 20

?

No.

Why?

Because every future number that we add is the same possible next-row value.

For example, suppose the next row is:

[1, 2, 3]

Then:

3 + 1 = 4
5 + 1 = 6
5 + 1 = 6
10 + 1 = 11
15 + 1 = 16
20 + 1 = 21

The larger sums can never suddenly become part of the first 3 results while smaller sums are available.

Therefore:

We only need to preserve the smallest k partial sums.

This is the central optimization.

4. Why use a max heap?

Now look at this:

```
priority_queue<int> pq;
```

By default, C++ `priority_queue` is a **max heap**.

So:

pq =

```
    10
   /  \
  7    8
```

The largest element is always available using:

```
pq.top()
```

Why do we want the largest?

Because we want to maintain exactly the **smallest k values**.

Suppose:

k = 3

Current heap:

3
5
8

Now we get:

4

The 3 smallest should become:

3, 4, 5

The largest among our current 3 is:

8

So we remove 8:

```
pq.pop();
```

and insert 4:

```
pq.push(4);
```

Now:

```
3, 4, 5
```

That's exactly what we want.

5. Why not use a min heap?

A min heap gives us the smallest value:

```
pq.top()
```

But when maintaining the **smallest k elements**, we need to quickly remove the **largest among those k elements**.

Therefore:

```
Max heap
  ↓
largest of current k
  ↓
easy to remove
```

That's why:

```
priority_queue<int> pq;
```

is used.

6. Now let's understand the code

Your code:

```
class Solution {
public:
    int kthSmallest(vector<vector<int>>& mat, int k) {

        vector<int> sums = mat[0];

        for (int i = 1; i < mat.size(); i++) {

            priority_queue<int> pq;

            for (int sum : sums) {

                for (int x : mat[i]) {

                    int newSum = sum + x;

                    if (pq.size() < k) {
                        pq.push(newSum);
                    }
                    else if (newSum < pq.top()) {
                        pq.pop();
                        pq.push(newSum);
                    }
                }
            }

            sums.clear();

            while (!pq.empty()) {
                sums.push_back(pq.top());
                pq.pop();
            }

        }

        return *max_element(sums.begin(), sums.end());
    }
};
```

```
}  
};
```

Let's go line by line.

Step 1: Start with first row

```
vector<int> sums = mat[0];
```

Suppose:

```
mat[0] = [1, 3, 5]
```

Then:

```
sums = [1, 3, 5]
```

Why?

Because after processing only the first row, possible sums are simply the elements of the first row.

7. Process remaining rows

```
for (int i = 1; i < mat.size(); i++)
```

We start from row 1 because row 0 is already inside sums .

For every new row, we generate:

```
previous partial sum + current row element
```

8. Create max heap

```
priority_queue<int> pq;
```

This heap will store:

The smallest k sums generated while processing this row.

9. Iterate through previous sums

```
for (int sum : sums)
```

Suppose:

```
sums = [3, 5, 5]
```

We take:

```
sum = 3
```

then:

```
sum = 5
```

then:

```
sum = 5
```

10. Iterate through current row

```
for (int x : mat[i])
```

Suppose:

```
mat[i] = [7, 8, 9]
```

For:

```
sum = 3
```

we generate:

```
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12
```

For:

```
sum = 5
```

we generate:

```
12
13
14
```

and so on.

11. Insert into heap

```
if (pq.size() < k) {
    pq.push(newSum);
}
```

Suppose:

```
k = 3
```

Initially:

```
pq = {}
```

Insert:

```
10
```

then:

```
pq = {10}
```

Insert:

```
11
```

then:

```
pq = {11, 10}
```

Insert:

```
12
```

then:

```
pq = {12, 10, 11}
```

Now heap size is 3 .

12. What happens when heap already has k elements?

Now suppose:

```
pq = {12, 10, 11}
```

and we encounter:

```
newSum = 9
```

We check:

```
else if (newSum < pq.top())
```

pq.top() is:

```
12
```

So:

```
9 < 12
```

True.

Therefore:

```
pq.pop();  
pq.push(9);
```

Now heap contains:

```
11  
10  
9
```

So we replaced the largest of our current k values with a smaller value.

13. What if newSum is larger?

Suppose:

```
pq = {12, 10, 11}
```

and:

```
newSum = 15
```

Then:

```
15 < pq.top()
```

means:

```
15 < 12
```

False.

So we do nothing.

Why?

Because:

```
15
```

cannot be one of the smallest 3 values when we already have:

```
10, 11, 12
```

14. After processing the row

We have:

```
sums.clear();
```

We delete the old partial sums.

Then:

```
while (!pq.empty()) {  
    sums.push_back(pq.top());  
    pq.pop();  
}
```

We transfer our k smallest values from the heap into `sums`.

So conceptually:

Before:

```
sums  
↓  
partial sums after row i-1
```

Generate combinations

↓

priority_queue

↓

smallest k sums

↓

sums

↓

partial sums after row i

This repeats for every row.

15. Why `max_element` at the end?

At the end:

```
return *max_element(sums.begin(), sums.end());
```

Remember that `sums` contains the smallest `k` final sums.

Therefore:

```
sums = [10, 11, 12]
```

If we need:

```
k = 3
```

the answer is:

```
12
```

which is the **largest among the smallest k values**.

Therefore:

```
max_element(...)
```

gives the k-th smallest.

16. Complete code

```
class Solution {
public:
    int kthSmallest(vector<vector<int>>& mat, int k) {
        vector<int> sums = mat[0];

        for (int i = 1; i < mat.size(); i++) {
            priority_queue<int> pq;

            for (int sum : sums) {
                for (int x : mat[i]) {
                    int newSum = sum + x;

                    if (pq.size() < k) {
                        pq.push(newSum);
                    }
                    else if (newSum < pq.top()) {
                        pq.pop();
                        pq.push(newSum);
                    }
                }
            }

            sums.clear();

            while (!pq.empty()) {
                sums.push_back(pq.top());
                pq.pop();
            }
        }
    }
};
```

```
    }  
    }  
  
    return *max_element(sums.begin(), sums.end());  
    }  
};
```

17. Why this approach is better than brute force

Suppose there are m rows and each row has n elements.

If we generate **every possible sum**, the number of combinations is:

$n \times n \times n \times \dots \times n$
 m times

which is:

n^m

For:

$n = 50$
 $m = 5$

that's:

$50^5 = 312,500,000$

Huge.

Our approach says:

"I don't need all possible sums. I only need the smallest k ."

So after each row:

number of sums $\leq k$

This is a massive reduction when k is small.

18. Now compare with Binary Search

There is another famous approach:

Binary search on the answer.

The idea is completely different.

Instead of generating sums, binary search asks:

"How many possible sums are $\leq X$?"

For example:

$X = 20$

We count:

number of sums ≤ 20

If count $\geq k$:

answer ≤ 20

so:

high = 20

Otherwise:

answer > 20

so:

```
low = 21
```

Eventually:

```
low == high
```

and that's the answer.

19. Heap vs Binary Search

The important distinction is:

Heap approach

We explicitly generate candidate sums:

```
previous k sums
      +
current row
      ↓
new candidates
      ↓
keep smallest k
```

Binary search

We don't generate sums.

Instead:

```
Guess X
      ↓
Count sums <= X
      ↓
Is count >= k?
      ↓
Move left/right
```

20. Which is better?

It depends on the constraints.

For this **k-th smallest sum** problem, the heap approach is often very intuitive when **k** is relatively small because we cap the candidate set at **k**.

Its complexity is approximately:

```
O(m × k × n × log k)
```

where:

- m** = number of rows
- n** = number of elements per row
- k** = requested rank

Memory:

```
O(k)
```

The binary-search approach can be better when:

```
k
```

is large, because it avoids maintaining **k** candidate sums, but it requires an efficient way to **count how many sums ≤ mid**. That counting step is usually more complicated.

So the practical tradeoff is:

Approach	Main idea	Complexity	Difficulty
Brute force	Generate everything	$O(n^m)$	Easy
Heap	Keep only k smallest	$O(mkn \log k)$	Medium
Binary search	Guess answer + count	Depends on counting	Harder

For learning and understanding this particular code, the heap approach is much easier to reason about: *generate candidates → keep only the best k → move to next row*.

21. Time complexity – carefully

Let's calculate it properly.

Assume:

```
m = number of rows
n = number of elements in each row
```

At every row:

```
for (int sum : sums)
```

There are at most:

```
k
```

elements in `sums`.

And:

```
for (int x : mat[i])
```

runs:

```
n
```

times.

Therefore, we generate:

```
k × n
```

candidates per row.

For each candidate, heap operation costs:

```
O(log k)
```

because the heap contains at most `k` elements.

Therefore:

```
O(k × n × log k)
```

per row.

There are `m - 1` rows.

Therefore:

```
O((m - 1) × k × n × log k)
```

which simplifies to:

```
O(m × n × k × log k)
```

Final time complexity:

```
O(mnk log k)
```

Space complexity:

The heap contains at most `k` elements:

```
O(k)
```

and `sums` also contains at most `k` elements:

```
O(k)
```

Therefore:

```
O(k)
```

extra space.

One important observation about the code

There is a small subtlety: the initial line

```
vector<int> sums = mat[0];
```

can contain n elements, not necessarily k .

So strictly, before the first reduction, `sums` has size n . If $n > k$, the first processed row generates $n \times n$ candidates rather than $k \times n$.

After that first iteration, `sums` is capped at k .

So the more precise complexity is:

```
First transition:  $O(n^2 \log k)$   
Remaining  $m-2$  transitions:  $O((m-2)nk \log k)$ 
```

giving:

```
 $O(n^2 \log k + mnk \log k)$ 
```

and in the usual simplified analysis:

```
 $O(mnk \log k)$ 
```

when treating n as bounded similarly across rows.

The core idea to remember for interviews is:

At every row, we only preserve the smallest k partial sums, because anything larger than those k candidates can never become part of the final k smallest sums. A max heap lets us efficiently discard the largest candidate whenever a better one arrives.